```
!pip -q install rdkit

import os
import random
import warnings
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from rdkit import Chem, RDLogger
from rdkit.Chem import AllChem
RDLogger.DisableLog('rdApp.*')

from sklearn.model_selection import StratifiedKFold, train_test_split
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    roc_auc_score, roc_curve
)
from sklearn.preprocessing import StandardScaler
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 36.7/36.7 MB 22.7 MB/s eta 0:00:00
```

```
SEED = 45
N_SPLITS = 5
VAL_SIZE_WITHIN_TEMP = 0.5  # temp set is split equally into val and test
RADIUS = 2
N_BITS = 2048

def set_seed(seed=42):
    random.seed(seed)
    np.random.seed(seed)

set_seed(SEED)

def smiles_to_morgan_fp(smiles_list, radius=2, n_bits=2048):
    fps = []
    for smi in smiles_list:
        mol = Chem.MolFromSmiles(smi)
        if mol is not None:
            fp = AllChem.GetMorganFingerprintAsBitVect(mol, radius, nBits=n_bits)
            fps.append(np.array(fp, dtype=np.float32))
        else:
            fps.append(np.zeros(n_bits, dtype=np.float32))
    return np.array(fps, dtype=np.float32)

def build_model(random_state):
    from sklearn.ensemble import RandomForestClassifier
    return RandomForestClassifier(
        n_estimators=100,
        random_state=random_state,
        n_jobs=-1
    )

def maybe_scale(X_train, X_val, X_test):
    return X_train, X_val, X_test


def evaluate_fold(model, X_test, y_test):
    y_pred = model.predict(X_test)

    if hasattr(model, "predict_proba"):
        y_score = model.predict_proba(X_test)[:, 1]
    else:
        y_score = model.decision_function(X_test)

    return {
        "accuracy": accuracy_score(y_test, y_pred),
        "precision": precision_score(y_test, y_pred, zero_division=0),
        "recall": recall_score(y_test, y_pred, zero_division=0),
        "f1": f1_score(y_test, y_pred, zero_division=0),
        "roc_auc": roc_auc_score(y_test, y_score)
    }, y_score
```

```
df = pd.read_csv("https://raw.githubusercontent.com/McahitKutsal/hivcsv/main/HIV7.csv")
df = df.sample(frac=1.0, random_state=SEED).reset_index(drop=True)

X = smiles_to_morgan_fp(df["smiles"], radius=RADIUS, n_bits=N_BITS)
y = df["HIV_active"].astype(int).values

skf = StratifiedKFold(n_splits=N_SPLITS, shuffle=True, random_state=SEED)
```

```
fold_results = []
roc_curves = []

for fold, (train_idx, temp_idx) in enumerate(skf.split(X, y), start=1):
    print(f"\n===== FOLD {fold} / {N_SPLITS} =====")
    set_seed(SEED + fold)

    X_train_full, y_train_full = X[train_idx], y[train_idx]
    X_temp, y_temp = X[temp_idx], y[temp_idx]

    X_val, X_test, y_val, y_test = train_test_split(
        X_temp, y_temp,
        test_size=VAL_SIZE_WITHIN_TEMP,
        stratify=y_temp,
        random_state=SEED + fold
    )

    X_train, X_val, X_test = maybe_scale(X_train_full, X_val, X_test)

    model = build_model(random_state=SEED + fold)
    model.fit(X_train, y_train_full)

    metrics, y_score = evaluate_fold(model, X_test, y_test)
    fold_results.append({
        "fold": fold,
        "test_accuracy": metrics["accuracy"],
        "test_precision": metrics["precision"],
        "test_recall": metrics["recall"],
        "test_f1": metrics["f1"],
        "test_roc_auc": metrics["roc_auc"]
    })

    fpr, tpr, _ = roc_curve(y_test, y_score)
    roc_curves.append((fold, fpr, tpr, metrics["roc_auc"]))

    print({k: round(v, 4) if isinstance(v, float) else v for k, v in fold_results[-1].items()})

results_df = pd.DataFrame(fold_results)
results_df
```

```
===== FOLD 1 / 5 =====
{'fold': 1, 'test_accuracy': 0.8512, 'test_precision': 0.9268, 'test_recall': 0.7703, 'test_f1': 0.8413, 'test_ro

===== FOLD 2 / 5 =====
{'fold': 2, 'test_accuracy': 0.8824, 'test_precision': 0.9672, 'test_recall': 0.7973, 'test_f1': 0.8741, 'test_ro

===== FOLD 3 / 5 =====
{'fold': 3, 'test_accuracy': 0.8581, 'test_precision': 0.935, 'test_recall': 0.777, 'test_f1': 0.8487, 'test_roc_

===== FOLD 4 / 5 =====
{'fold': 4, 'test_accuracy': 0.9135, 'test_precision': 0.9692, 'test_recall': 0.8571, 'test_f1': 0.9097, 'test_ro

===== FOLD 5 / 5 =====
{'fold': 5, 'test_accuracy': 0.8201, 'test_precision': 0.9204, 'test_recall': 0.7075, 'test_f1': 0.8, 'test_roc_a
```

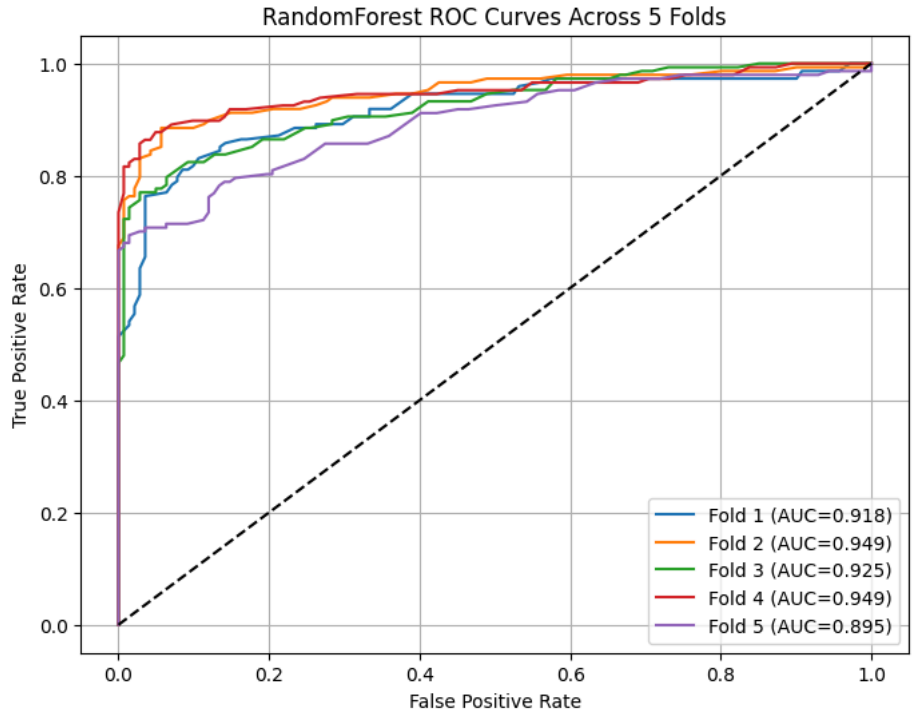
	fold	test_accuracy	test_precision	test_recall	test_f1	test_roc_auc
0	1	0.851211	0.926829	0.770270	0.841328	0.918200
1	2	0.882353	0.967213	0.797297	0.874074	0.948989
2	3	0.858131	0.934959	0.777027	0.848708	0.925029
3	4	0.913495	0.969231	0.857143	0.909747	0.949267
4	5	0.820069	0.920354	0.707483	0.800000	0.895348

```
summary_rows = []
for metric in ["test_accuracy", "test_precision", "test_recall", "test_f1", "test_roc_auc"]:
    mean_val = results_df[metric].mean()
    std_val = results_df[metric].std(ddof=1)
    var_val = results_df[metric].var(ddof=1)
    summary_rows.append({
        "Metric": metric.replace("test_", "").upper(),
        "Mean": mean_val,
        "Std": std_val,
        "Variance": var_val,
        "Formatted": f"{mean_val:.4f} ± {std_val:.4f}"
    })

summary_df = pd.DataFrame(summary_rows)
summary_df
```

	Metric	Mean	Std	Variance	Formatted
0	ACCURACY	0.865052	0.035032	0.001227	0.8651 ± 0.0350
1	PRECISION	0.943717	0.022971	0.000528	0.9437 ± 0.0230
2	RECALL	0.781844	0.053841	0.002899	0.7818 ± 0.0538
3	F1	0.854772	0.040658	0.001653	0.8548 ± 0.0407
4	ROC_AUC	0.927367	0.022704	0.000515	0.9274 ± 0.0227

```
plt.figure(figsize=(8, 6))
for fold, fpr, tpr, auc_val in roc_curves:
    plt.plot(fpr, tpr, label=f"Fold {fold} (AUC={auc_val:.3f})")
plt.plot([0, 1], [0, 1], "k--")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("RandomForest ROC Curves Across 5 Folds")
plt.legend()
plt.grid(True)
plt.show()
```




```
results_df.to_csv("RandomForest_fold_results.csv", index=False)
summary_df.to_csv("RandomForest_summary_results.csv", index=False)

print("Saved:")
print("- RandomForest_fold_results.csv")
print("- RandomForest_summary_results.csv")
```

```
Saved:
- RandomForest_fold_results.csv
- RandomForest_summary_results.csv
```